

II.- What your Assignment Report must include:

1. Step by step geometry solution (5 points)

Outer rectangle(white+blue part) = $2A \times 2B$

Inner rectangle(blue_A) = $A \times B$

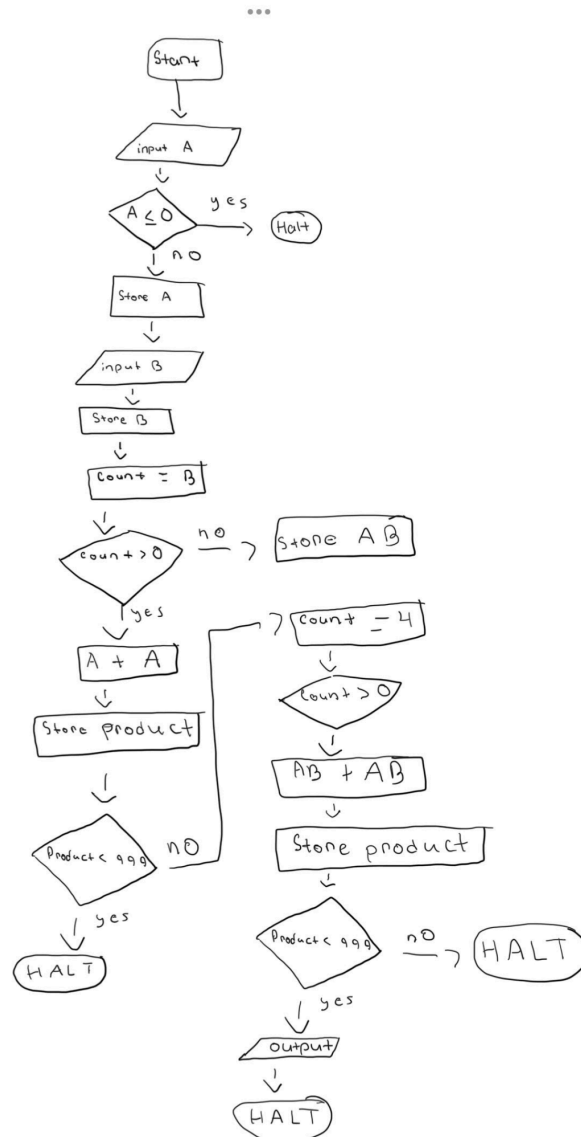
White part = total-blue_area

= $(2A \times 2B) - (A \times B)$

= $4AB - AB$

white_A = $3AB$

2. A flowchart showing the logic flow of your program. (15 points)



3. The LMC program (in Word) to resolve the problem with the appropriate comments (15 points)

```

BRA MAIN
RET BRA 000 // return to calling code, to be modified by calling code

```

```
MUL LDA MUL0 // multiply mul0 with mul1 and set OVERFLOW if overflow
BRZ RET //branch to return when input=0
LDA MUL1 //load 2nd number
BRZ RET //when 2nd num = 0, return to start
LDA MUL1 //load 2nd number
SUB C1 //countdown each loop
STA MUL1
LDA MAXVAL //load max value (999)
SUB MULRES //sub current product from 999 (finds range until 999)
SUB MUL0 //maxval-mult0
BRP MULNOOVERFLOW //if no overflow, add
LDA C1 //load 1 to C1 when no overflow
STA OVERFLOW
BRA RET //return when overflow
```

```
MULNOOVERFLOW LDA MULRES
ADD MUL0 //add mul0 to mulres
STA MULRES //store value
BRA MUL //repeat loop until it reaches 0
```

```
MULRES DAT //holds result of product
MUL0 DAT //1st num
MUL1 DAT //2nd num
```

```
MAIN INP //user input 1st number
BRZ EXIT //when input = 0, end program
STA MUL0 //store 1st input in mul0
INP //input and store 2nd number
STA MUL1
LDA C0 //load 0 before multiplication
STA MULRES //product starts at 0
STA OVERFLOW //clear overflow
LDA RETA0 //prepares to find 4AB by going to next line
STA RET // set returning address
BRA MUL //jump to multiplication
```

```
LDA MULRES //load result of first multiplication (A*B)
STA AB //store result in AB
STA MUL0 //store in mul0
LDA C4 //count = 4
STA MUL1 //4AB = mul1
LDA C0 //clear product
STA MULRES
LDA RETA1 //prepare to compute 3AB
STA RET // set returning address
BRA MUL //jump to multiplication
```

```
LDA OVERFLOW //check for overflow
BRZ NOOVERFLOW //overflow = 0(no overflow), continue
LDA C0 //0 is loaded into accumulator
```

```
SUB C1 //sub 1 from result when overflow reached
OUT
BRA MAIN //restart program

NOOVERFLOW LDA AB //load result
STA MUL0 //store into mul0
LDA C3 //load 3 to find 3AB
STA MUL1 //set up to find 3AB

LDA C0 //clear accumulator before next calculation
STA MULRES //reset product
LDA RETA2 //return address
STA RET // set returning address to find 3AB and restart
BRA MUL //branch to multiply to find 3AB

LDA MULRES //load final result
OUT //output the result
BRA MAIN //restart program
EXIT HLT //end program

AB DAT
C0 DAT 0
C1 DAT 1
C2 DAT 2
C3 DAT 3
C4 DAT 4
RETA0 DAT 634 //line 34
RETA1 DAT 644 //line 44
RETA2 DAT 659 //line 59
OVERFLOW DAT
MAXVAL DAT 999
```

4. Description of two test scenarios explaining the results (15 points):

Initial

Assembly Language Code

```

MULRES DAT 20 DAT 00
MUL0 DAT 21 DAT 00
MUL1 DAT 22 DAT 00
MAIN INP // main function
      BRZ EXIT
      STA MUL0
      INP
      STA MUL1
      LDA C0
      STA MULRES
      STA OVERFLOW
      LDA RETA0
      STA RET // set
returning address
      BRA MUL
      LDA MULRES
      STA AB
      STA MUL0
      LDA C4
      STA MUL1
      LDA C0
      STA MULRES
      LDA RETA1
      STA RET // set
returning address
      BRA MUL
      LDA OVERFLOW
      BRZ NOOVERFLOW
      LDA C0
      SUB C1
      OUT
      BRA MAIN
NOOVERFLOW LDA AB
      STA MUL0
      LDA C3
      STA MUL1
      LDA C0
      STA MULRES
  
```

OUTPUT

CPU

- PROGRAM COUNTER: 00
- INSTRUCTION REGISTER
- ADDRESS REGISTER
- ACCUMULATOR: 000
- ARITH-METIC UNIT

INPUT

RAM

0	1	2	3	4	5	6	7	8	9
623	600	521	701	522	701	522	265	322	573
10	11	12	13	14	15	16	17	18	19
220	221	816	565	372	601	520	121	320	602
20	21	22	23	24	25	26	27	28	29
000	000	000	901	762	321	901	322	564	320
30	31	32	33	34	35	36	37	38	39
372	569	301	602	520	363	321	568	322	564
40	41	42	43	44	45	46	47	48	49
320	570	301	602	572	750	564	265	902	623
50	51	52	53	54	55	56	57	58	59
563	321	567	322	564	320	571	301	602	520
60	61	62	63	64	65	66	67	68	69
902	623	000	000	000	001	002	003	004	634
70	71	72	73	74	75	76	77	78	79
644	659	000	999	000	000	000	000	000	000
80	81	82	83	84	85	86	87	88	89
000	000	000	000	000	000	000	000	000	000
90	91	92	93	94	95	96	97	98	99
000	000	000	000	000	000	000	000	000	000

CONTROL

ASSEMBLE INTO RAM RUN STEP

RESET LOAD HELP SELECT

INPUT: 01

OUTPUT: 02

RUN/STEP your program, SELECT, LOAD or edit program

OPTIONS

© Mike Coley and Peter Higginson

> One test scenario that shows the logic for when the area of the outer rectangle is bigger than 999

- Given the input 50 and 25 (= 1250, will be invalid)
- Once this is calculated, the program will detect overflow since the maximum value the program will hold is 999

Final

Assembly Language Code

```

00 BRA 23
01 BRA 00
02 LDA 21
03 BRZ 01
04 LDA 22
05 BRZ 01
06 LDA 22
07 SUB 65
08 STA 22
09 LDA 73
10 SUB 20
11 SUB 21
12 BRP 16
13 LDA 65
14 STA 72
15 BRA 01
16 LDA 20
17 ADD 21
18 STA 20
19 BRA 02
20 DAT 00
21 DAT 00
22 DAT 00
23 INP
24 BRZ 62
25 STA 21
26 INP
27 STA 22
28 LDA 64
29 STA 20
30 STA 72
31 LDA 69
32 STA 01
33 BRA 02
34 LDA 20
35 STA 63
36 STA 21
37 LDA 68
38 STA 22
39 STA 22 // set

```

CPU

- PROGRAM COUNTER: 24
- INSTRUCTION REGISTER: 9
- ADDRESS REGISTER: 01
- ACCUMULATOR: -001
- ARITH-METIC UNIT

OUTPUT

-1

INPUT

01

RAM

0	1	2	3	4	5	6	7	8	9
623	644	521	701	522	701	522	265	322	573
10	11	12	13	14	15	16	17	18	19
220	221	816	565	372	601	520	121	320	602
20	21	22	23	24	25	26	27	28	29
950	950	002	901	762	321	901	322	564	320
30	31	32	33	34	35	36	37	38	39
372	569	301	602	520	363	321	568	322	564
40	41	42	43	44	45	46	47	48	49
320	570	301	602	572	750	564	265	902	623
50	51	52	53	54	55	56	57	58	59
563	321	567	322	564	320	571	301	602	520
60	61	62	63	64	65	66	67	68	69
902	623	000	950	000	001	002	003	004	634
70	71	72	73	74	75	76	77	78	79
644	659	001	999	000	000	000	000	000	000
80	81	82	83	84	85	86	87	88	89
000	000	000	000	000	000	000	000	000	000
90	91	92	93	94	95	96	97	98	99
000	000	000	000	000	000	000	000	000	000

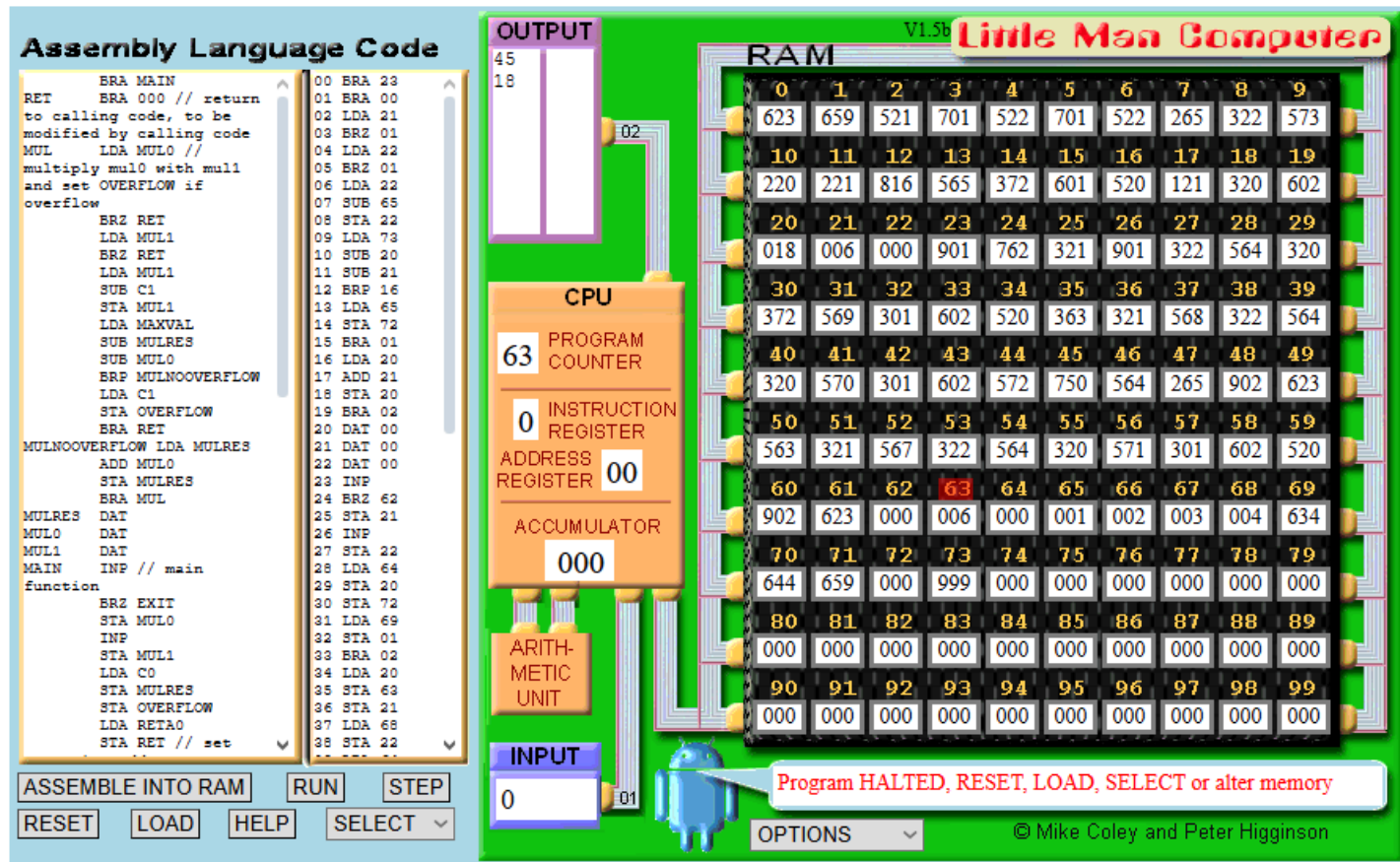
Buttons: ASSEMBLE INTO RAM, STOP, <<, >>, RESET, LOAD, HELP, SELECT, SPEED 6

INPUT required

Options: OPTIONS

© Mike Coley and Peter Higginson

> One test scenario where the program loops 2 times before it is stopped with the 000 Final



- Input for first two loops: A = 5, B = 3, A = 2, B = 3
- Last loop: A = 0
- After the first loop, the program will output 3AB which in the first cause is 45
- Once this is outputted, the user will be prompted for input
- In the second loop, the program will output 24 (3AB)
- On the third loop, when inputted 0, the program branches to end and stops

> Show a screenshot of the initial and the final screens for each scenario.

III. Short video in MP4 format showing yourself running the simulator. You should include in your video (50 points):

- Show the program in the simulator and explain of the logic of your program
- Show the program execution of the two test scenarios in the simulator

c) What happens when you manually replace the value in the mailbox containing 902 to the value 922 and run your program again? An explanation of what is that output value now. (Hint: Look into ASCII characters)

- 901 = user input
- 902 = output
- Causes error since 922 is not an instruction for these tasks
- 922 does not mean anything to LMC

Work Cited

LMC Instruction Set, <https://www.yorku.ca/sychen/research/LMC/LMCInstructions.html>.